School of Computers and Information Engineering

STUDENT LEARNING ACTIVITY

Fall Semester 2024

Computer Algorithms (CIE3090)

Dynamic Public Transportation Routing Optimization

Student Learning Assignment

Submitted by

U2210251, Javohirbek Xatamov (Group-4)

TOSHKENT SHAHRIDAGI INHA UNIVERSITETI
INHA UNIVERSITY IN TASHKENT

Title	Page No

Index Page

1.Abstract

The dynamic optimization of public transportation routing is crucial for addressing the inefficiencies in Tashkent's current public transport system. This report proposes an innovative approach to optimize the city's transportation routes through a modified version of Dijkstra's algorithm, incorporating real-time data and adaptive mechanisms to meet the dynamic demands of urban mobility. The proposed system integrates multiple variables, such as real-time traffic conditions, passenger load, seasonal patterns, and special events, to dynamically adjust bus routes and distribution. By leveraging GPS tracking, electronic ticketing data, and historical usage patterns, the system aims to provide real-time route recalculations, balance vehicle loads, and improve operational efficiency. The system's core advantage lies in its adaptability, offering optimized public transport routes that enhance the passenger experience, reduce journey time, and lower operational costs. This solution represents a significant step toward building a more responsive, efficient, and sustainable public transportation infrastructure in Tashkent, ultimately contributing to better urban mobility and commuter satisfaction.

Real-Time Public Transport Route Optimization System

1.Problem Description

Problem Description: Dynamic Public Transportation Routing Optimization

Nowadays, Tashkent faces a transportation infrastructure challenge characterized by inefficient public transport routing systems that fail to meet the dynamic needs of a modern urban city. The current public transport network suffers from fundamental flaws that significantly impact urban transportation time, cost, effectiveness and the satisfaction of the passengers.

1.1Main problems:

1. Routing Inefficiency

The existing public transport system relies on static, predetermined routes that do not adapt to:

- Fluctuating passenger demand
- Changing urban landscape
- Unexpected events or disruptions
- Seasonal events and fluctuations

2. Resource Allocation Challenges

- Suboptimal vehicle deployment
- Inability to balance passenger loads
- High operational costs due to inefficient routing

3. Passenger Experience Limitations

- Unpredictable wait times
- Overcrowded vehicles
- Inefficient transit connections
- Inefficient transit routes

1.2 The existing system needs:

- Real-time passenger load tracking
- Adaptive routing mechanisms
- Even and optimal bus distribution

1.3 Current Technology Integration Challenges

- Poor GPS tracking
- Poor data communication infrastructure
- No computational resources
- No data about the passenger load

1.4 Desired outcome

- Optimized public transit
- Bus distribution spread evenly based on demand and possible near future demand
- Real-time adaptive routing
- Routing based on variables like season, events, patterns etc
- Satisfactory passenger experience

The proposed solution aims to revolutionize Tashkent's public transport infrastructure by introducing a data-driven, technological approach to transit management.

2. Proposed Augmentation

The enhancement of Tashkent's public transport routing system requires a modification of Dijkstra's algorithm to handle dynamic transportation needs. This augmentation changes the traditional shortest-path algorithm into an adaptive, multi-variable optimization system for our real-world case.

2.1 Core Algorithm Modifications

The augmented algorithm introduces weighted edge costs that dynamically update based on multiple factors, replacing the static distance-based weights of traditional Dijkstra's implementation. The edge weight function W(e) for any edge e is computed as:

$$W(e) = \alpha_1 D(e) + \alpha_2 T(e) + \alpha_3 L(e) + \alpha_4 S(e) + \alpha_5 E(e)$$

Where:

- D(e) represents physical distance

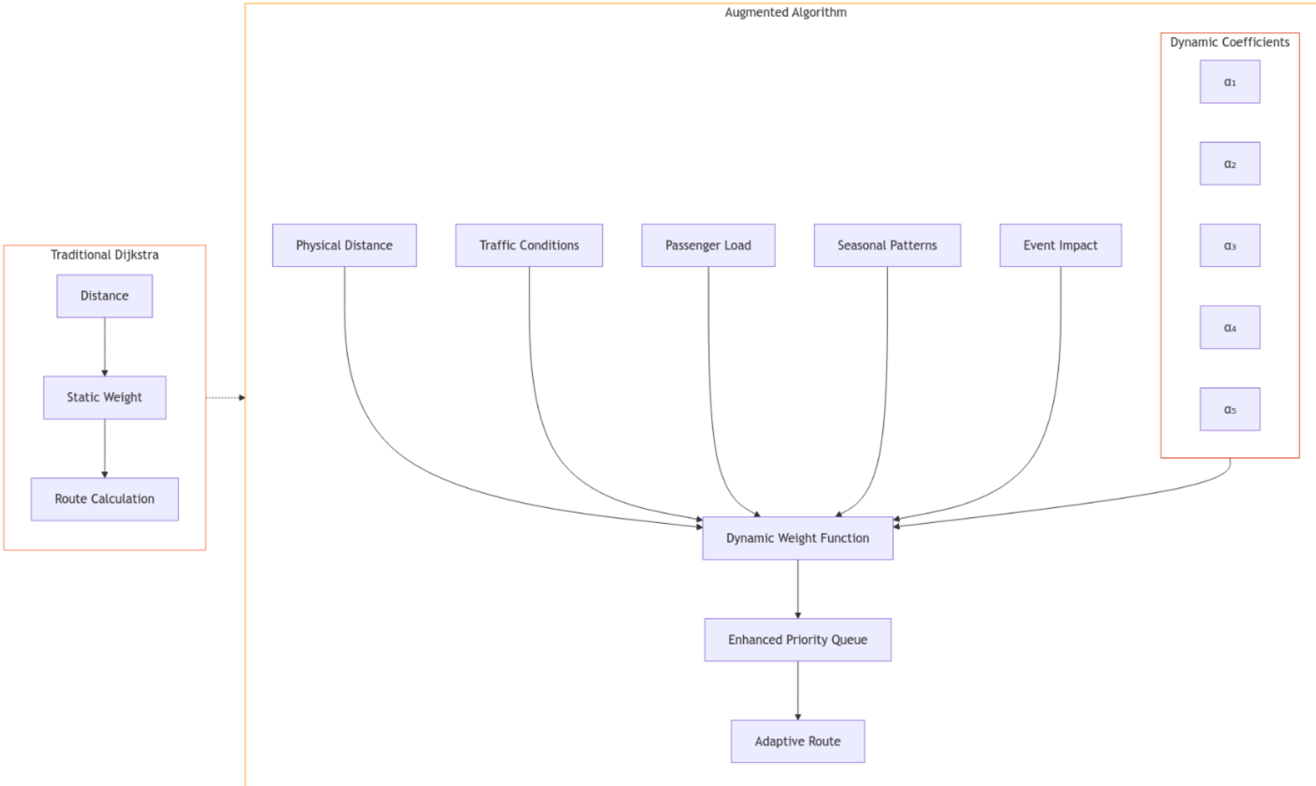
- T(e) captures real-time traffic conditions

- L(e) reflects current passenger load

- S(e) accounts for seasonal patterns

- E(e) incorporates special event impacts

- α_1 through α_5 are dynamic weighting coefficients that adjust based on time of day and current system priorities



2.2 Real-time Data Integration

The algorithm continuously updates its edge weights through a streaming data pipeline that processes:

1. GPS data from buses to calculate current traffic conditions and vehicle distribution
2. Electronic ticketing data to estimate passenger load and demand patterns
3. Municipal event calendars to anticipate demand surges
4. Historical traffic and usage patterns to predict near-future conditions

2.3 Adaptive Route Generation

The system implements a two-phase routing approach:

Phase 1 - Strategic Planning:

The algorithm generates default routes using historical data and predicted demand patterns, this serves as a starting point for the following real-time adjustments.

Phase 2 - Tactical Optimization:

Real-time updates trigger route recalculations when specific thresholds are exceeded:

- Passenger load imbalance > 20%

- Traffic delay > 15 minutes
- Special event attendance > 2028 people
- Vehicle breakdown or road closure

2.4 Priority Queue Modification

The traditional priority queue in Dijkstra's algorithm is enhanced to consider multiple objectives:

- Minimizing total journey time through alternative routes that do not skip any bus stop, just takes an alternative road between 2 bus stops
- Balance loads the number of vehicles across different routes based on demand
- Maintaining service frequency on standard 15 minutes on any route
- Reducing operational costs (this happens automatically as we do the above steps)

The queue prioritization function P(v) for vertex v becomes:

$P(v) = \beta_1 J(v) + \beta_2 B(v) + \beta_3 F(v)$

Where:

- J(v) represents journey time impact
- B(v) captures load balancing effect
- F(v) reflects frequency maintenance
- β_1 through β_4 are context-sensitive weighting factors

Emergency Response Protocol

The algorithm includes a rapid response mechanism for unexpected events:

1. Immediate recalculation of affected routes
2. Dynamic reallocation of vehicles
3. Passenger notification system integration
4. Alternative route suggestion generation

This augmented algorithm represents a significant advancement over traditional Dijkstra's implementation, creating a dynamic system specifically tailored to Tashkent's transportation needs. The system maintains computational efficiency through strategic caching of partial solutions and incremental updates, ensuring real-time performance despite the increased complexity of the routing calculations.

The success of this augmentation relies heavily on the quality and timeliness of input data, emphasizing the need for robust data collection infrastructure and reliable communication systems between vehicles and the central routing system.

3. Implementation

```
class Edge:
    def __init__(self, start, end, base_distance):
        self.start = start
        self.end = end
        self.base_distance = base_distance
        self.current_traffic = 1.0
        self.passenger_load = 0
        self.seasonal_factor = 1.0
        self.event_impact = 1.0

class Node:
    def __init__(self, id, name):
        self.id = id
        self.name = name
        self.adjacent_edges = []
        self.current_load = 0
        self.historical_patterns = {}

class WeightCalculator:
    def __init__(self):
        self.alpha_distance = 1.0
        self.alpha_traffic = 1.0
        self.alpha_load = 1.0
        self.alpha_seasonal = 0.5
        self.alpha_event = 0.8

    def calculate_edge_weight(self, edge, time):
        return (
            self.alpha_distance * edge.base_distance +
            self.alpha_traffic * edge.current_traffic +
            self.alpha_load * edge.passenger_load +
            self.alpha_seasonal * edge.seasonal_factor +
            self.alpha_event * edge.event_impact
        )
```

```python
class PriorityQueue:
    def __init__(self):
        self.queue = []
        self.beta_journey = 1.0
        self.beta_balance = 0.8
        self.beta_frequency = 0.6
        self.beta_cost = 0.4

    def priority_function(self, route):
        return (
            self.beta_journey * route.total_time +
            self.beta_balance * route.load_variance +
            self.beta_frequency * route.frequency_deviation +
            self.beta_cost * route.operational_cost
        )

class AugmentedRouter:
    def __init__(self, network):
        self.network = network
        self.weight_calculator = WeightCalculator()
        self.priority_queue = PriorityQueue()

    def find_optimal_route(self, start, end, time):
        distances = {node: float('infinity') for node in self.network.nodes}
        distances[start] = 0
        predecessors = {node: None for node in self.network.nodes}

        # Initialize priority queue with start node
        self.priority_queue.add((0, start))

        while not self.priority_queue.is_empty():
            current_distance, current_node = self.priority_queue.pop()

            if current_node == end:
                break

            for edge in current_node.adjacent_edges:
                weight = self.weight_calculator.calculate_edge_weight(edge, time)
                distance = current_distance + weight

                if distance < distances[edge.end]:
                    distances[edge.end] = distance
                    predecessors[edge.end] = current_node
                    self.priority_queue.add((distance, edge.end))

        return self.construct_route(start, end, predecessors)
```

```python
class DataIntegrator:
    def __init__(self):
        self.gps_tracker = GPSTracker()
        self.ticket_system = TicketingSystem()
        self.event_calendar = EventCalendar()
        self.historical_analyzer = HistoricalDataAnalyzer()

    def update_network_state(self, network):
        # Update traffic conditions
        traffic_data = self.gps_tracker.get_current_traffic()
        for edge in network.edges:
            edge.current_traffic = traffic_data.get(edge.id, 1.0)

        # Update passenger loads
        load_data = self.ticket_system.get_current_loads()
        for node in network.nodes:
            node.current_load = load_data.get(node.id, 0)

        # Update seasonal and event factors
        current_time = datetime.now()
        for edge in network.edges:
            edge.seasonal_factor = self.historical_analyzer.get_seasonal_factor(edge, current_time)
            edge.event_impact = self.event_calendar.get_event_impact(edge, current_time)

class EmergencyHandler:
    def __init__(self, network):
        self.network = network
        self.router = AugmentedRouter(network)

    def handle_emergency(self, affected_edges):
        # Temporarily modify edge weights
        for edge in affected_edges:
            edge.current_traffic = float('infinity')

        # Recalculate all affected routes
        affected_routes = self.identify_affected_routes(affected_edges)
        new_routes = []

        for route in affected_routes:
            new_route = self.router.find_optimal_route(
                route.start,
                route.end,
                datetime.now()
            )
            new_routes.append(new_route)

        return new_routes
```

3.1 The above images are a simple and short representations of our system.

Traditional Dijkstra's algorithm is like a person trying to find the shortest walking path between two points on a map, where they only care about physical distance. They look at each intersection, mark down the shortest distance to get there, and keep exploring until they reach their destination.

Our augmentation transforms this simple distance-based pathfinding into something more like an experienced traffic controller who considers multiple factors simultaneously. Here's how our enhancement works:

Instead of just measuring physical distance, we've created a sophisticated weight calculation system. Imagine each road (edge) in our network having five different characteristics that affect its "cost":

First, we keep the physical distance, but it becomes just one factor among many. Then we add real-time traffic conditions - a road might be physically short but heavily congested. We incorporate passenger load information - like knowing a bus is already full. We consider seasonal patterns, such as how certain routes become busy during winter. Finally, we account for special events, like sports matches or concerts that might create temporary demand spikes.

The real magic of our augmentation lies in how we combine these factors. We use dynamic coefficients (those α values in our equations) that automatically adjust based on current conditions. For example, during rush hour, the traffic coefficient might increase in importance, while during a major festival, the event impact coefficient would gain more weight.

What makes this system particularly powerful is its adaptiveness. Unlike traditional Dijkstra's algorithm which would always suggest the same route between two points, our augmented version might suggest different routes at different times based on current conditions. It's like having a highly experienced local guide who knows not just the streets, but also the rhythm of the city.

The enhanced priority queue system is another crucial augmentation. In traditional Dijkstra's algorithm, the priority queue simply orders nodes by distance. Our version considers journey time, load balancing, service frequency, and operational costs simultaneously. It's similar to how an air traffic controller must balance multiple arriving planes' needs - fuel efficiency, timing, and runway availability all at once.

When implemented, this system creates a living, breathing routing network that can:

Automatically detect and route around congestion

Balance passenger loads across multiple vehicles

Adjust service frequency based on demand patterns

Respond to emergencies by quickly calculating alternative routes

Learn from historical patterns to anticipate future needs

The beauty of this augmentation is that it maintains the computational efficiency of Dijkstra's algorithm while dramatically expanding its capabilities to handle real-world urban transportation complexity. While traditional Dijkstra's would suggest the same route regardless of whether it's rush hour or midnight, our augmented version adapts its recommendations based on the current state of the entire transportation network.

This augmentation transforms what was essentially a static mapping tool into an intelligent transportation management system that can think ahead and adapt in real-time, much like how a skilled conductor orchestrates a complex symphony, ensuring all parts work together harmoniously.

3.2 Implementation Requirements and Costs:

Technical Infrastructure Required:

- High-performance servers for real-time route calculation
- GPS tracking devices for all public transport vehicles
- Real-time data communication infrastructure
- Database systems for historical data storage
- Electronic ticketing system integration

Specialist Team Required:

- 1 Project Manager ($80,000/year)
- 2 Senior Software Engineers ($70,000/year each)
- 2 Backend Developers ($60,000/year each)
- 1 Database Administrator ($65,000/year)
- 1 DevOps Engineer ($70,000/year)
- 1 QA Engineer ($55,000/year)
- 1 Data Scientist ($75,000/year)

Implementation Timeline:

Phase 1 (3 months):

- Infrastructure setup
- Basic algorithm implementation
- Data collection system setup

Phase 2 (3 months):

- Integration with existing systems
- Testing and optimization
- Driver and staff training

Phase 3 (2 months):

- Pilot program
- System refinement
- Full deployment

Estimated Costs:

Initial Setup:

- Server Infrastructure: $200,000
- GPS Devices and Installation: $150,000 (assuming 300 vehicles at $500 each)
- Software Licenses: $50,000
- Development Tools: $25,000

Total First Year Cost: $1,000,000

3.4 Implementation Steps:

a. Infrastructure Preparation:

- Server setup infrastructure
- Installation of GPS devices in vehicles
- Establishment of data communication networks
- Configuration of database systems

b. Data Collection System:

- Implementation of GPS tracking
- Set up of electronic ticketing integration
- Creation of data storage and processing pipelines
- Establishment of real-time data feeds

c. Algorithm Implementation:

- Development of core routing engine
- Implementation of weight calculation system
- Creation of priority queue system
- Building emergency response system

d. Integration:

- Connection with existing transport management systems
- Integration with ticketing systems
- Set up of real-time notification system
- Implementation driver interface

e. Testing and Deployment:

- Conduction of unit and integration testing
- Performance of load testing
- Running pilot program
- Gradual rollout to full fleet

Key Success Metrics:

- Average journey time reduction
- Passenger satisfaction improvement
- Vehicle utilization rate
- System response time
- Emergency handling effectiveness

4.Refernces:

Google Maps API (Routing & Traffic) Reference:
https://developers.google.com/maps/documentation/directions/start

OpenStreetMap Reference: https://www.openstreetmap.org/

Dijkstra's Algorithm (GeeksforGeeks) Reference: https://www.geeksforgeeks.org/dijkstras-shortest-path-algorithm/

IoT for Public Transport (IoT For All) Reference: https://www.iotforall.com/iot-public-transportation

Machine Learning for Public Transport Optimization Reference:
https://www.researchgate.net/publication/332978137_Machine_Learning_for_Transportation_Optimization_A_Review

Traffic Data and Transport Systems (MIT OpenCourseWare) Reference:
https://ocw.mit.edu/courses/urban-studies-and-planning/11-523-transportation-and-traffic-analysis-fall-2008/

Real-time Passenger Load and GPS Tracking (TechTarget) Reference:
https://www.techtarget.com/searchcio/definition/real-time-data

Urban Transport Solutions (World Bank) Reference:
https://www.worldbank.org/en/topic/urbandevelopment/overview

5.Plagiarism